

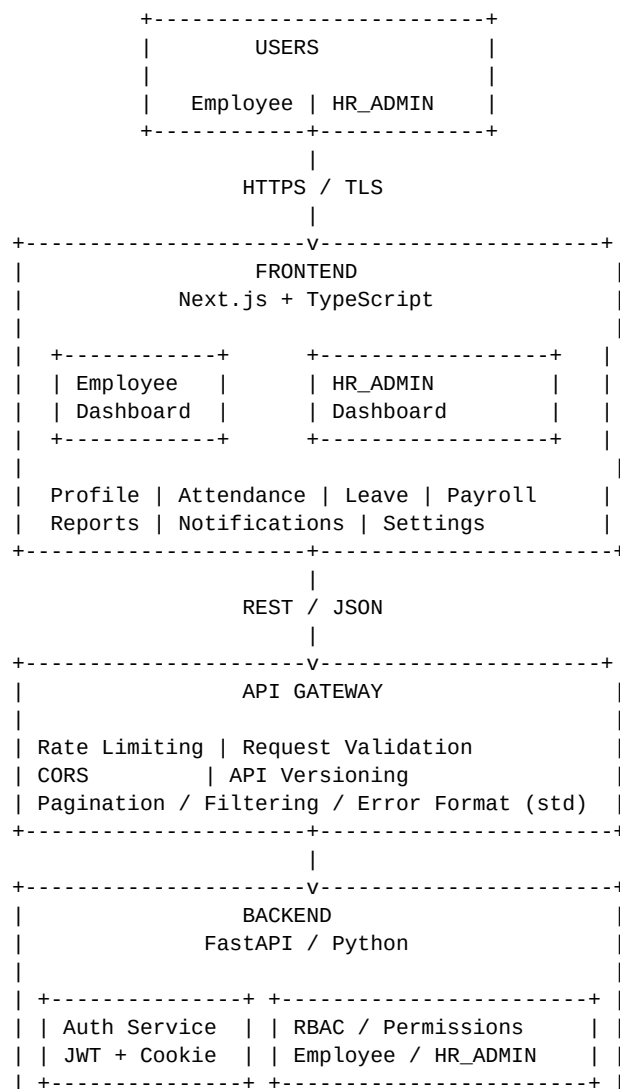
DAYFLOW — Final Hackathon Architecture

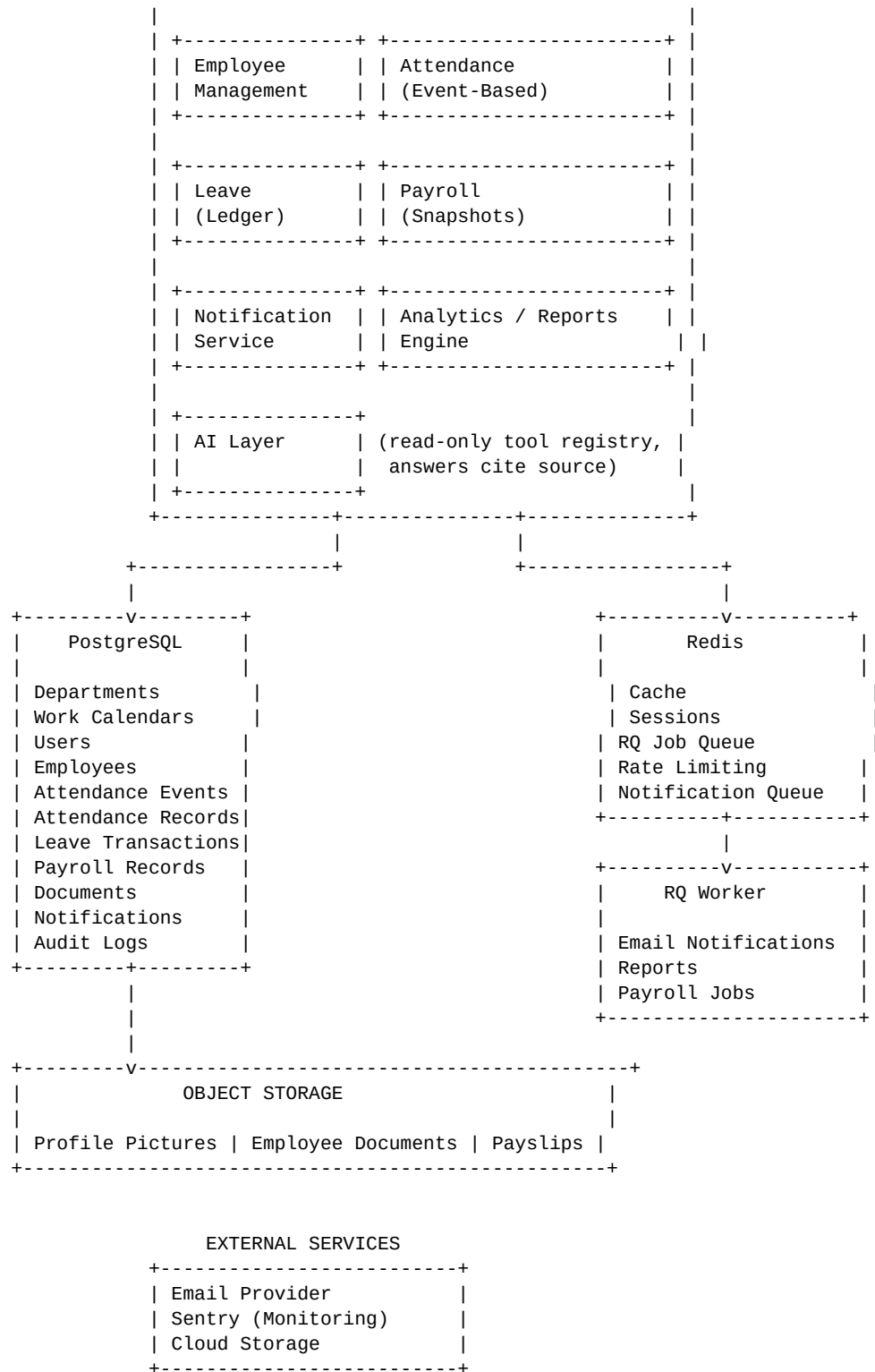
Frozen — Single-Company, Single-Stack Edition (v3)

If this is going to be presented as a hackathon project, the architecture below is the one to build. It is designed to look like a real HRMS product, not just CRUD pages. This version freezes every remaining open decision: roles are merged into a single privileged role, the technology stack has no "A or B" choices left, the system is explicitly single-company, and money, timestamps, idempotency and AI transparency are all specified concretely.

The uploaded requirements define two primary roles — Employee and Admin/HR — and core modules for authentication, profiles, attendance, leave, payroll, approvals, notifications and analytics. This document keeps that scope as the source of truth, and formally collapses "Admin" and "HR" into one backend role, `HR\_ADMIN`, since the requirements never actually require them to behave differently.

System Diagram





Dayflow is built for a single company. There is no organization table and no `organization\_id` anywhere in the schema — every table belongs directly to the one company the system is deployed for. If multi-company support is ever needed, that is a deliberate future migration, not something the hackathon schema pretends to support today.

1. The Architecture I Would Actually Build

Frontend — Next.js + TypeScript

```
src/
|
+-- app/
|   +-- login/
|   +-- register/
|   +-- dashboard/
|   |
|   +-- employee/
|   |   +-- profile/
|   |   +-- attendance/
|   |   +-- leave/
|   |   +-- payroll/
|   |   +-- notifications/
|   |
|   +-- admin/
|   |   +-- dashboard/
|   |   +-- employees/
|   |   +-- attendance/
|   |   +-- leave/
|   |   +-- payroll/
|   |   +-- reports/
|   |   +-- audit/
|   |
+-- components/
+-- hooks/
+-- services/
+-- lib/
+-- types/
```

Employee and HR_ADMIN aren't merely different pages. They are different permission contexts, enforced on the backend, never assumed from the frontend route alone. The `/admin` route tree is shown to any authenticated `HR\_ADMIN` user — there is no separate "HR" tree, since HR and Admin are the same backend role.

2. Backend Architecture

Use a modular monolith, not microservices. That remains the right choice for a hackathon team — it avoids the operational overhead of microservices while still giving clean module boundaries.

```
backend/
|
+-- app/
|   |
|   +-- main.py
|   |
|   +-- core/
|   |   +-- config.py
|   |   +-- security.py
|   |   +-- database.py
|   |   +-- permissions.py
|   |
|   +-- auth/
|   |   +-- router.py
|   |   +-- service.py
|   |   +-- schemas.py
```

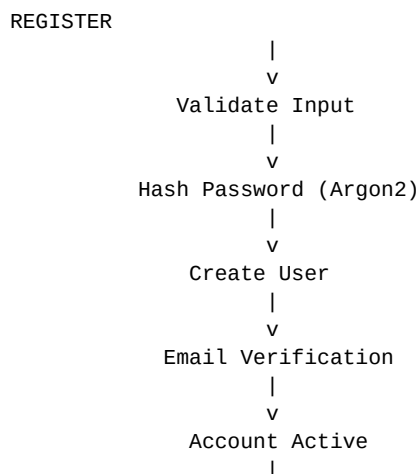
```

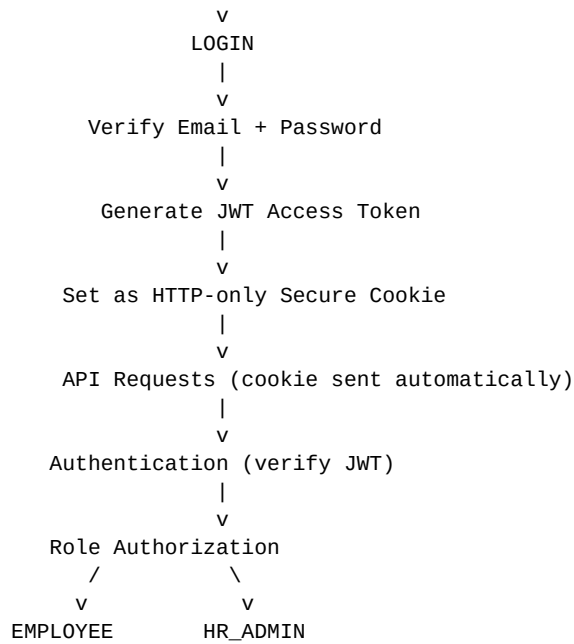
| | +-- models.py
| |
| +-- employees/
| | +-- router.py
| | +-- service.py
| | +-- schemas.py
| | +-- models.py
| |
| +-- attendance/
| | +-- router.py
| | +-- service.py
| | +-- schemas.py
| | +-- models.py
| |
| +-- leave/
| | +-- router.py
| | +-- service.py
| | +-- schemas.py
| | +-- models.py
| |
| +-- payroll/
| | +-- router.py
| | +-- service.py
| | +-- schemas.py
| | +-- models.py
| |
| +-- notifications/
| |
| +-- reports/
| |
| +-- audit/
| |
| +-- ai/
| |
| +-- workers/
|
+-- tests/

```

3. Authentication Architecture (Simplified)

Authentication is intentionally kept simple: a single JWT access token, stored in an HTTP-only secure cookie. There is no separate refresh-token issuance/rotation system for the hackathon — the access token has a moderate lifetime, and re-login is required after expiry.





The requirements explicitly require email verification, secure passwords and login-based dashboard redirection.

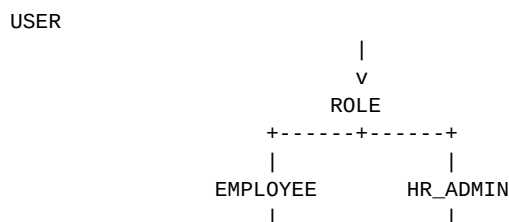
Internal identity: `users.id` is a UUID and is the true internal primary key for a person. It is intentionally kept separate from the human-facing employee identifier (see Section 5).

Security

- Argon2 password hashing
- JWT access token in an HTTP-only, Secure, SameSite cookie
- RBAC (Employee / HR_ADMIN)
- Input validation
- Rate limiting
- CORS
- Audit logging
- Environment-based secrets

4. RBAC Architecture

HR and Admin are merged into a single backend role, `HR\_ADMIN`, since the requirements never actually distinguish their permissions. There is no separate "HR" role in the database or the permission checks — only `EMPLOYEE` and `HR\_ADMIN`.



State transitions must be concurrency-safe and idempotent — enforced through database locks and unique constraints, not just application-level checks.

4B. Formal State Machines

```
Leave:
PENDING
  +-- APPROVED
  +-- REJECTED
APPROVED
  +-- CANCELLED (optional)

Payroll:
DRAFT -> GENERATED -> PUBLISHED

Employee:
ACTIVE -> INACTIVE
(no hard delete – see Section 5)

Attendance:
OPEN -> COMPLETED
```

Gate — you should be able to prove:

- Employee cannot access another employee's data
- Employee cannot approve leave
- Employee cannot modify payroll
- HR_ADMIN can manage employees, approve leave, and view/modify payroll

5. Database Architecture

Use PostgreSQL. Dayflow is single-company: there is no `organizations` table and no `organization\_id` column anywhere. Departments, employees, and every other table belong directly to the one deployed company.

5.1 Identity model

`users.id` (internal identity, UUID) is kept separate from `employees.employee\_code` (business-facing identifier such as `EMP-1024`). Since Dayflow is single-company, `employee\_code` is simply globally `UNIQUE` — no company-scoping is needed.

```
users
-----
id UUID PK
email
password_hash
role          -- 'EMPLOYEE' | 'HR_ADMIN'
is_active
email_verified
created_at
updated_at

employees
-----
id UUID PK
user_id FK
employee_code UNIQUE
work_calendar_id FK
department_id FK
position_id FK
first_name
```

```

last_name
phone
address
profile_picture
joining_date
is_active          -- deactivation flag, see 5.4
created_at
updated_at

```

All primary keys across the schema use UUIDs rather than predictable sequential integers, with the human-friendly `employee\_code` (e.g. `EMP-1024`) used for display.

5.2 Work calendar linkage (new)

Every employee is explicitly linked to a work calendar, which is where timezone and working-day configuration lives, since there is no organization table to hold that configuration globally:

```

work_calendars
-----
id UUID PK
name
timezone
working_days      -- e.g. ['MON', 'TUE', 'WED', 'THU', 'FRI']
work_start_time
work_end_time
created_at
updated_at

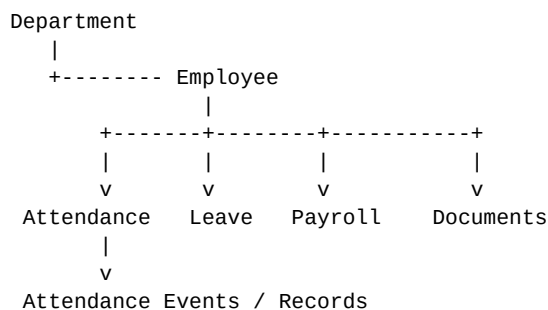
holidays
-----
id UUID PK
work_calendar_id FK
date
name
created_at

employees.work_calendar_id -> work_calendars.id

```

For a single-company hackathon deployment there will typically be exactly one `work\_calendars` row (e.g. "Default — Asia/Kolkata"), but every employee still references it explicitly rather than the timezone being hardcoded, so a second calendar (e.g. a remote/international employee) can be added without a schema change.

5.3 Core entities



5.4 Deactivation instead of deletion (frozen rule)

Employees are never hard-deleted. Removing someone from active duty sets `employees.is_active = false`` (the `ACTIVE -> INACTIVE`` transition from Section 4B). This is a deliberate, permanent rule: attendance, leave, and payroll history must remain intact and queryable for a deactivated employee, both for audit purposes and because payroll/leave records reference `employee_id`` via foreign key.

5.5 Money fields (frozen rule)

Every salary/money field in the schema uses `NUMERIC`` (Python `Decimal``), never floating point:

<code>salary_structures.basic_salary</code>	<code>NUMERIC(12,2)</code>
<code>salary_structures.allowances</code>	<code>NUMERIC(12,2)</code>
<code>salary_structures.deductions</code>	<code>NUMERIC(12,2)</code>
<code>payroll_records.basic_salary</code>	<code>NUMERIC(12,2)</code>
<code>payroll_records.allowances</code>	<code>NUMERIC(12,2)</code>
<code>payroll_records.deductions</code>	<code>NUMERIC(12,2)</code>
<code>payroll_records.gross_salary</code>	<code>NUMERIC(12,2)</code>
<code>payroll_records.net_salary</code>	<code>NUMERIC(12,2)</code>

Floating-point types are never used for money, to avoid rounding errors in salary calculations.

5.6 Core tables

```
departments
job_positions

users
employees

work_calendars
holidays

attendance_events
attendance_records

leave_types
leave_transactions
leave_requests

salary_structures
payroll_records

documents
notifications

audit_logs
```

5.7 Timestamps (frozen rule)

Every mutable table includes `created_at`` and `updated_at`` columns, maintained consistently (`updated_at`` refreshed on every write). This applies to `users``, `employees``, `attendance_records``, `leave_requests``, `leave_transactions``, `salary_structures``, `payroll_records``, `documents``, `notifications``, and `work_calendars``. Append-only tables (`attendance_events``, `audit_logs``) only need `created_at``, since they are never updated in place.

5.8 Indexes

```
users(email) UNIQUE

employees(employee_code) UNIQUE
employees(department_id)
employees(work_calendar_id)

attendance_records(employee_id, date) UNIQUE
attendance_events(employee_id, timestamp)

leave_requests(employee_id, status)
leave_requests(start_date, end_date)

notifications(user_id, read_at)

audit_logs(actor_id, created_at)
audit_logs(entity_type, entity_id)

payroll_records(employee_id, year, month) UNIQUE
```

5.9 Constraints

```
employee_code UNIQUE
email UNIQUE
basic_salary >= 0
allowances >= 0
deductions >= 0
leave_days > 0
start_date <= end_date

attendance_records:
(employee_id, date) UNIQUE

payroll_records:
(employee_id, year, month) UNIQUE
```

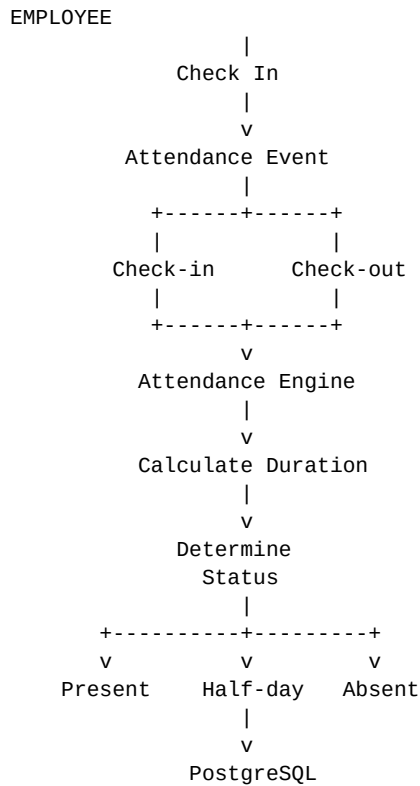
5.10 Important relationships

```
users
  1
  |
  1
employees
  |
  +----- N ----- attendance_records
  |
  +----- N ----- leave_requests
  |
  +----- 1 ----- salary_structure
  |
  +----- N ----- payroll_records
  |
  +----- N ----- documents
  |
  +----- 1 ----- work_calendar
```

6. Attendance Architecture (Event-Based)

Instead of simply storing `employee\_id`, `date`, `status`, attendance is event-based: every check-in and check-out is recorded as an immutable event, and the daily attendance record is derived from those events.

This is a practical event log for the attendance domain — not a claim of full event-sourcing across the whole system.



Your requirements already define daily/weekly views, check-in/check-out and Present, Absent, Half-day and Leave states.

6.1 Events vs derived records

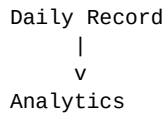
```

attendance_events  -- raw, immutable, one row per check-in/check-out
-----
id
employee_id
event_type        -- CHECK_IN | CHECK_OUT
timestamp
source
metadata

attendance_records -- derived, one row per employee per day
-----
employee_id
date
first_check_in
last_check_out
worked_minutes
status
late_minutes
early_leave_minutes

Raw Events
  |
  v
Attendance Engine
  |
  v

```



Rule: `attendance\_events` are never edited or deleted after creation; `attendance\_records` are always derived/recomputed from them.

6.2 Timezone and working calendar

Attendance is always interpreted relative to the employee's linked `work\_calendar\_id` (Section 5.2) — timezone, working days, and holidays all come from that calendar, not from a hardcoded assumption. This governs midnight boundaries, late/early calculations, monthly attendance percentage, and holiday handling.

6.3 Hackathon-level additions

- Late arrival detection
- Early departure
- Working hours
- Attendance percentage
- Monthly attendance summary

6.4 Idempotent check-in/out

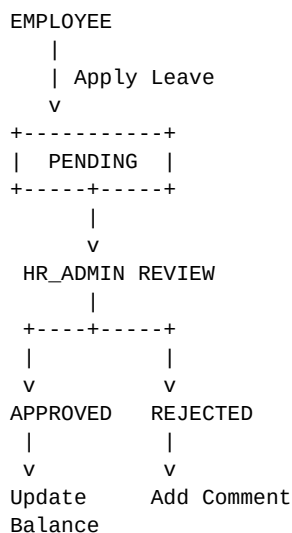
See Section 4A — duplicate check-in/check-out requests (double-click, network retry) do not create duplicate `attendance\_events` rows.

6.5 Views

Employee: Today, This Week, This Month. HR_ADMIN: All Employees, Department, Date, Status.

Gate — test: check-in, check-out, duplicate check-in, duplicate check-out, missing checkout, leave day, half-day.

7. Leave Management Workflow



```

|
v
Employee Notification

```

The source defines leave types as Paid, Sick and Unpaid, with Pending/Approved/Rejected states and HR/Admin (now `HR\_ADMIN`) approval or rejection.

7.1 Overlap validation — transaction/concurrency-safe (frozen)

```

Employee
|
v
New Leave Request
|
v
BEGIN TRANSACTION
|
v
Lock employee's existing leave rows (SELECT ... FOR UPDATE)
|
v
Check existing approved/pending leave for date overlap
|
+-- Overlap -> ROLLBACK, reject with LEAVE_OVERLAP
|
+-- No overlap -> Insert request, COMMIT

```

The overlap check and the insert happen inside the same database transaction, with a row lock on the employee's existing leave requests, so two near-simultaneous submissions for overlapping dates cannot both succeed. Without this, an employee could submit Aug 25–28 and separately Aug 27–30, producing overlapping approved leave.

The schema also leaves room for policy fields, even if not all are implemented for the hackathon:

```

minimum_notice_days
maximum_consecutive_days
weekend_policy
holiday_policy

```

7.2 Leave balance as a ledger

Rather than a simple mutable `allocated / used / remaining` structure, leave balance is tracked as a ledger for auditability:

```

leave_transactions
-----
employee
leave_type
amount
transaction_type
reference_id
created_at

+12 ALLOCATION
-2  LEAVE_USED
+2  LEAVE_CANCELLED
+1  MANUAL_ADJUSTMENT

Remaining = Allocations - Usage + Adjustments

```

Remaining can still be cached for performance, but the ledger gives full history — directly answering "why does this employee have only 4 days remaining?"

7.3 Approval transaction (idempotent)

Don't directly modify the employee's leave balance from the frontend. Run it as a single database transaction, guarded per Section 4A so a repeated approve/reject click cannot double-apply:

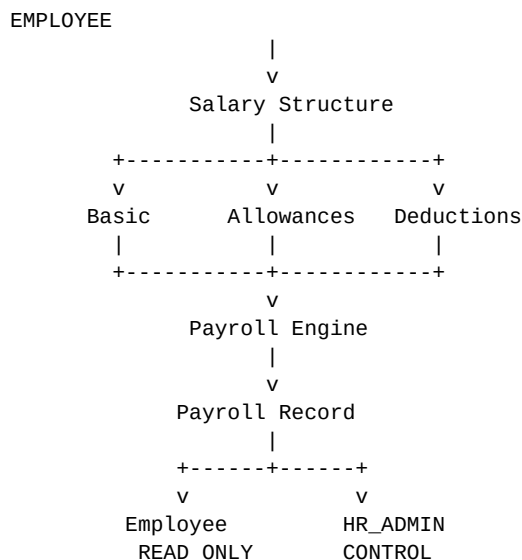
```
Leave Request
|
Backend Validation
|
Lock request row (must still be PENDING)
|
Approval Transaction
|
Leave Balance Update (ledger entry)
|
Notification
|
Audit Log

BEGIN TRANSACTION
Lock leave request (SELECT ... FOR UPDATE)
If status != PENDING: ROLLBACK, reject
Update leave request
Update leave balance (ledger entry)
Create notification
Create audit record
COMMIT
```

7.4 Leave dashboards

Employee: Available leave, Pending, Approved, Rejected, History. HR_ADMIN: Pending requests, Employee, Department, Leave type, Date, Approve, Reject, Comment.

8. Payroll Architecture



The requirements specify read-only payroll visibility for employees and salary-structure modification/control for `HR\_ADMIN`.

8.1 Historical snapshots with explicit components (frozen)

`salary\_structures` (the current, editable configuration) and `payroll\_records` (a frozen snapshot per pay period) are explicitly separated, so changing an employee's current salary structure never alters an already-generated payroll record. Each `payroll\_records` row stores every component that went into it, not just the final number:

```
salary_structures
  |
  v
Payroll calculation
  |
  v
payroll_records
-----
id UUID PK
employee_id FK
year
month
basic_salary    NUMERIC(12,2)
allowances      NUMERIC(12,2)
deductions      NUMERIC(12,2)
gross_salary    NUMERIC(12,2)  -- basic + allowances
net_salary      NUMERIC(12,2)  -- gross - deductions
status          -- DRAFT | GENERATED | PUBLISHED
generated_at
created_at
updated_at

August 2026
Basic          = Rs 50,000
Allowance      = Rs 10,000
Deduction      = Rs 5,000
Gross          = Rs 60,000
Net            = Rs 55,000
```

Rule: `payroll\_records` are immutable historical snapshots, not live-computed values, and they store the full component breakdown (basic, allowances, deductions, gross, net) so a payslip can always be regenerated exactly as it was.

8.2 Idempotent generation

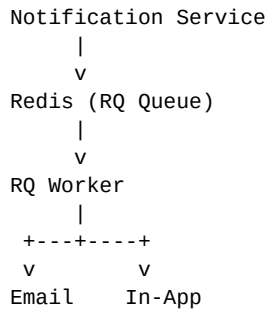
See Section 4A — generating payroll for an employee for a period that already has a `payroll\_records` row (enforced by the `(employee\_id, year, month)` unique constraint) returns the existing record rather than creating a duplicate.

For the hackathon, generate a salary-slip PDF using ReportLab and offer it as a download.

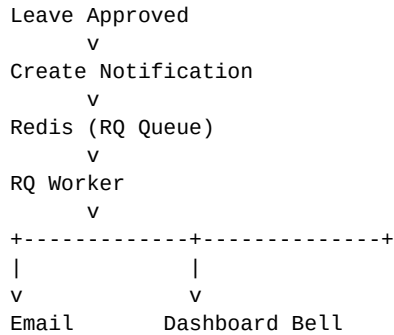
9. Notification Architecture

Don't send emails directly inside API requests. Background jobs run through ****RQ (Redis Queue)**** — this is the single frozen choice for job processing, not an "either/or" with Celery.

```
Application
  |
  v
```

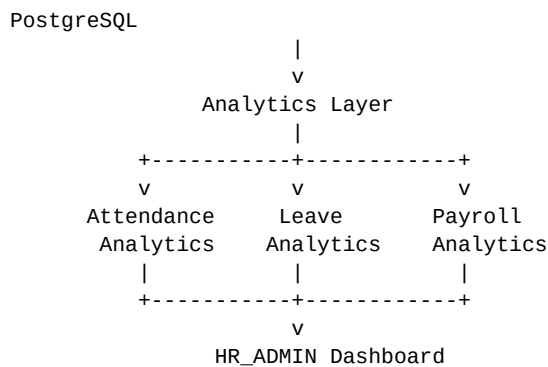


Example:



Email/notification alerts are listed as a future enhancement in the requirements, so this is a natural upgrade for the hackathon version.

10. Analytics Architecture



Dashboard:

HR ANALYTICS			
Employees	Attendance	On Leave	Payroll
248	94.2%	12	Rs XX,XX,XXX
Attendance Trend (bar chart)			
Leave Distribution		Department Attendance	

Sick	(bar)	Engineering	95%	
Paid	(bar)	HR	97%	
Unpaid	(bar)	Marketing	91%	
+-----+-----+				

Analytics/reporting is explicitly mentioned in the requirements as a future enhancement.

11. Audit System

Every important administrative action gets recorded, with request tracing so a failure can be followed end to end.

```
audit_logs
-----
id
actor_id
action
entity_type
entity_id
old_value
new_value
ip_address
request_id
success
reason
created_at
```

Example:

```
Actor: HR_ADMIN (id 1024)
Action: UPDATE_SALARY
Employee: 2045
Old: Rs 50,000
New: Rs 55,000
Time: 22 Aug 2026 09:14
```

```
Request
|
v
API
|
v
Business operation
|
v
Audit event
```

The `request\_id`, `success`/`failure`, and `reason` fields make it possible to trace a single request end-to-end, which is extremely useful when debugging after deployment.

12. API Architecture

Version the API from day one:

```
/api/v1/auth
/api/v1/users
/api/v1/employees
/api/v1/attendance
/api/v1/leaves
```

```
/api/v1/payroll
/api/v1/notifications
/api/v1/reports
/api/v1/audit
/api/v1/ai
```

Example:

```
POST    /api/v1/auth/login

GET      /api/v1/employees/me
PATCH   /api/v1/employees/me

POST     /api/v1/attendance/check-in
POST     /api/v1/attendance/check-out
GET      /api/v1/attendance/me

POST     /api/v1/leaves
GET      /api/v1/leaves/me

GET      /api/v1/admin/leaves
PATCH   /api/v1/admin/leaves/{id}/approve
PATCH   /api/v1/admin/leaves/{id}/reject

GET      /api/v1/payroll/me
GET      /api/v1/admin/payroll
PATCH   /api/v1/admin/payroll/{employee_id}
```

`/api/v1/admin/\*` routes require the `HR\_ADMIN` role — there is no separate `/hr/\*` route tree.

12.1 Pagination, filtering and error conventions

List endpoints accept:

```
?page=1
&page_size=20
&search=arman
&sort=-created_at
```

Example:

```
GET /api/v1/admin/employees
    ?page=1
    &page_size=20
    &search=engineering
```

Response shape:

```
{
  "items": [],
  "page": 1,
  "page_size": 20,
  "total": 248
}
```

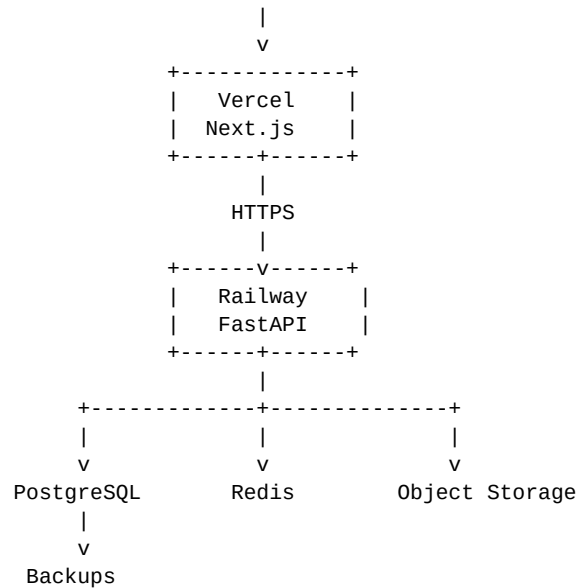
Errors are standardized:

```
{
  "code": "LEAVE_OVERLAP",
  "message": "Leave request overlaps an existing request."
}
```

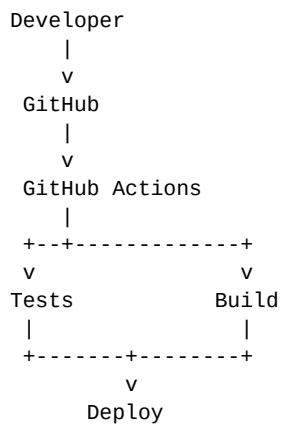
13. Deployment Architecture (Frozen)

Frontend -> Vercel
Backend -> Railway
PostgreSQL -> Railway
Redis -> Railway
Storage -> S3-compatible storage

INTERNET

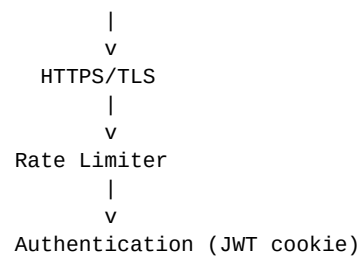


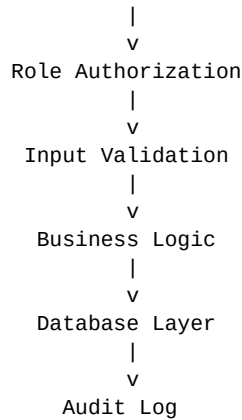
CI/CD



14. Security Architecture

REQUEST





Add:

- Argon2 password hashing
- JWT in an HTTP-only, Secure, SameSite cookie
- RBAC (Employee / HR_ADMIN)
- SQL injection protection through the ORM
- XSS protection
- CSRF protection (via SameSite cookie + CSRF token on state-changing requests)
- Rate limiting
- Secure HTTP headers
- Secrets in environment variables
- Database backups
- Audit logging

14.1 Observability from day one

```

structured logging
request ID
health endpoint
error tracking (Sentry)

```

Established from the start of the backend build, not bolted on at the end.

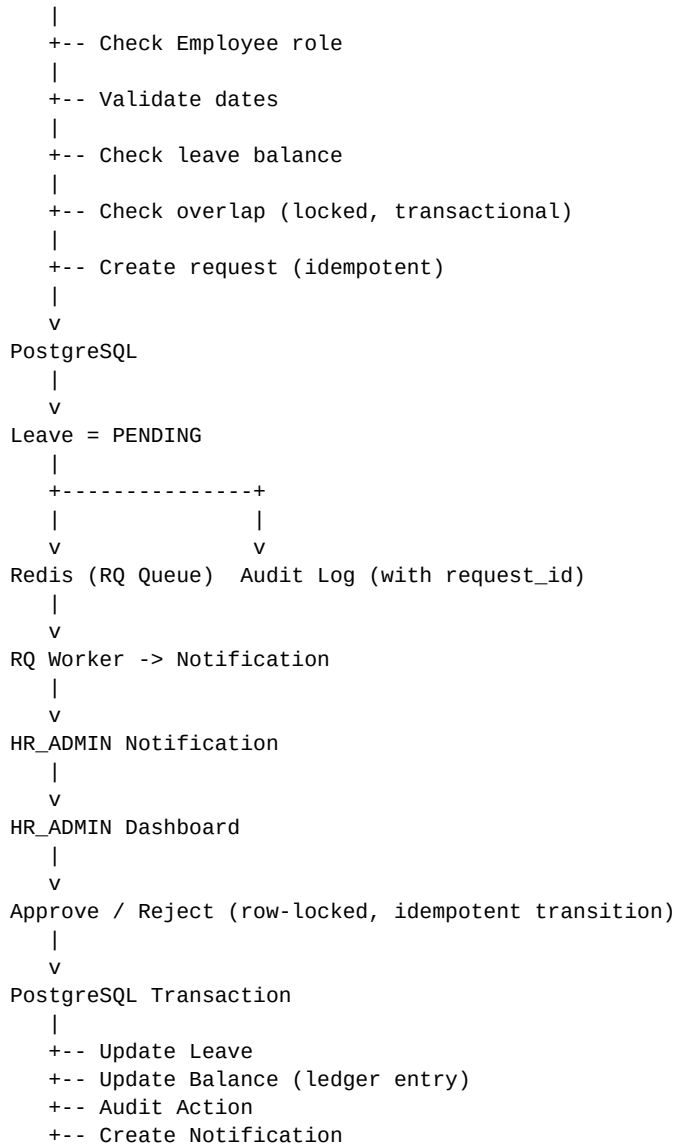
15. Complete Request Flow

Suppose an employee applies for leave.

```

Employee
  |
  v
Next.js UI
  |
  | POST /api/v1/leaves
  v
FastAPI
  |
  |-- Authenticate user (JWT cookie)

```



16. Final Technology Stack (Frozen — No Alternatives)

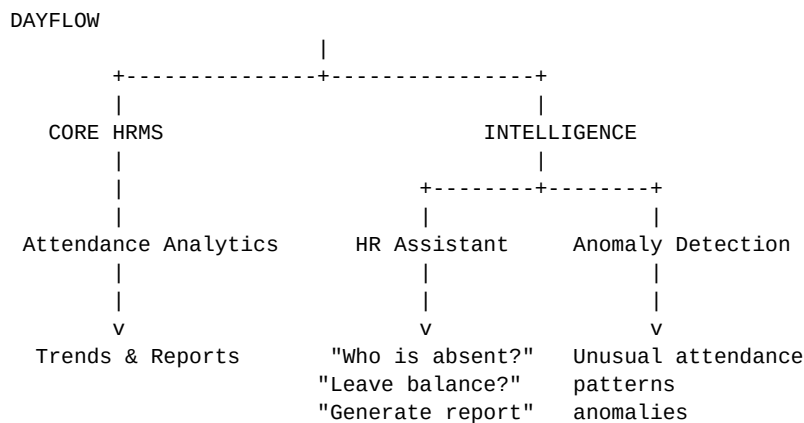
Layer	Choice
Frontend	Next.js + TypeScript
Styling	Tailwind CSS
Components	shadcn/ui
Backend	FastAPI + Python
API	REST + OpenAPI
Database	PostgreSQL
ORM	SQLAlchemy
Migrations	Alembic
Money fields	NUMERIC / Decimal (never float)
Authentication	JWT + HTTP-only Secure cookie
Password hashing	Argon2
Identifiers	UUID (primary keys) + human-readable employee_code
Cache / Queue	Redis
Background Jobs	RQ (Redis Queue)
File Storage	S3-compatible storage
Charts	Recharts

PDF	ReportLab
Testing	Pytest + Playwright
Logging/Monitoring	Structured logging + Sentry
CI/CD	GitHub Actions
Frontend Hosting	Vercel
Backend Hosting	Railway
Database Hosting	Railway (managed PostgreSQL)
Containerization	Docker

Every previously open "A or B" decision — background job runner, refresh-token design, hosting provider, password hashing algorithm — is now a single frozen choice.

17. The "Hackathon Killer" Layer (AI)

If you want Dayflow to be more than another HR CRUD project, add one intelligent layer after the core system works.



For example, an HR_ADMIN user could ask:

"Which departments have attendance below 90% this month?"

and Dayflow could query its own analytics data and return the result. Or:

"Show employees with unusually high absence this month."

The important thing is that AI should sit on top of the HRMS, not replace the core architecture.

17.1 Read-only by default



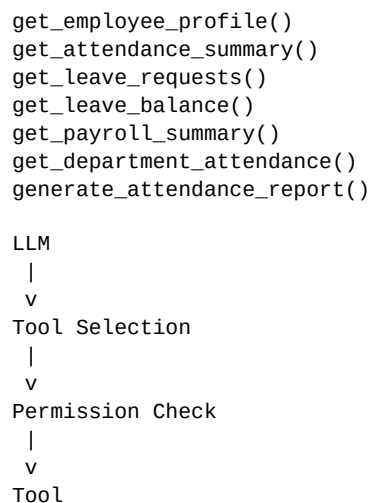
This is used instead of the unsafe shortcut `User -> LLM -> Database`. AI-generated actions are read-only by default:

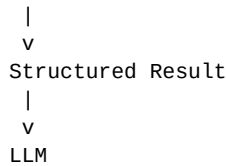
```
AI -> READ

never:
AI -> DELETE EMPLOYEE
AI -> MODIFY SALARY
AI -> APPROVE LEAVE
```

If write-capable AI actions are wanted later, they require explicit human confirmation before execution.

17.2 AI tool registry





17.3 Answer and source transparency (new)

Every AI answer states which tool it called and the underlying data it relied on, rather than presenting a bare conclusion. For example:

```

Question: "Which department has the lowest attendance this month?"

Answer:  "Marketing, at 91% average attendance this month."
Source:  tool = get_department_attendance()
        data = attendance_records, Aug 2026, 3 departments compared

```

This keeps the assistant auditable in the same spirit as the rest of the system: an HR_ADMIN user reading an AI answer can see exactly which tool produced it and which records it drew from, not just trust a generated sentence.

17.4 AI observability

```

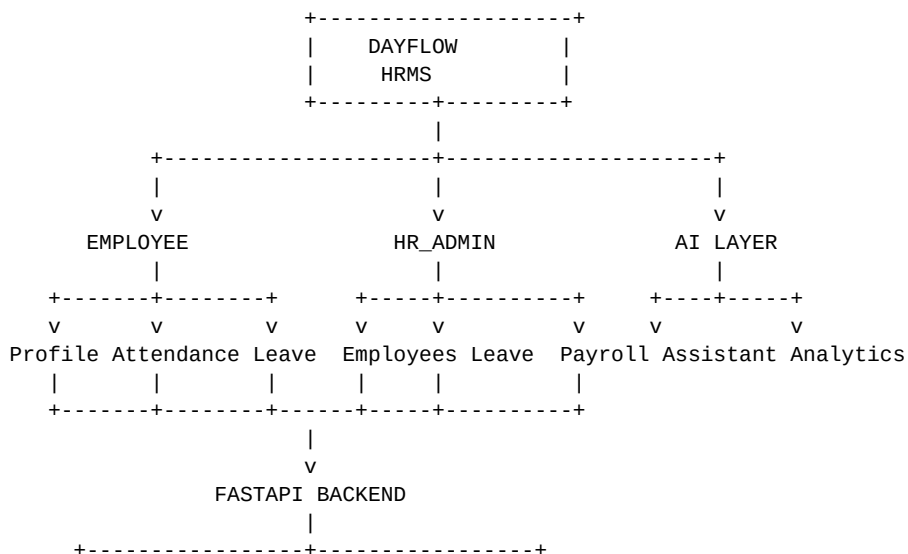
ai_requests
-----
user_id
request
tool_used
timestamp
latency
result_status

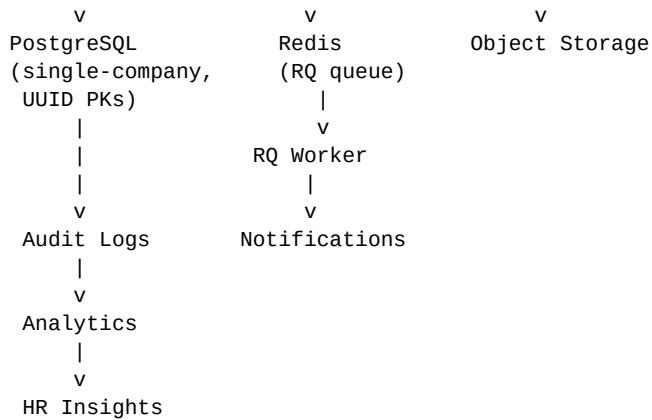
```

Sensitive raw data is not stored unnecessarily. This lets you demonstrate that every AI query is authorized, sourced, and traceable — a strong hackathon talking point.

Final Architecture

So the final project becomes:





This is the architecture I would freeze for the hackathon: a single-company modular monolith + PostgreSQL + Redis/RQ + Next.js, with one merged `HR\_ADMIN` role, JWT-cookie authentication, event-based attendance, ledgered leave balances with transaction-safe overlap checks, immutable payroll snapshots priced in `NUMERIC`, soft-deleted (never hard-deleted) employees, consistent timestamps, idempotent check-in/approval/payroll actions, audit trails with request tracing, and a guarded, read-only, source-citing AI layer. Every technology decision is now a single frozen choice — there are no remaining "or" options anywhere in this document.

Mandatory Before Coding Begins

1. Single-company schema — no `organizations` table, no `organization\_id` anywhere.
2. `HR\_ADMIN` as the one privileged role, replacing separate HR/Admin roles everywhere in code and UI.
3. `employees.work\_calendar\_id` wired to `work\_calendars`/`holidays` for timezone and working-day logic.
4. All money fields declared `NUMERIC`/`Decimal`; `payroll\_records` stores the full component breakdown (basic, allowances, deductions, gross, net) as an immutable snapshot.
5. Employee removal is always `is\_active = false`, never a delete.
6. `created\_at`/`updated\_at` present on every mutable table.
7. Leave overlap check and leave approval both run inside a locked database transaction.
8. Idempotency enforced concretely: attendance check-in/out (existing-open-event check), leave approval (row lock + status check), payroll generation (unique constraint on employee/year/month).
9. AI answers include the tool used and the data source, not just a conclusion.
10. Technology stack has zero remaining alternatives — Section 16 is the final word on every tool choice.

With these changes applied, the architecture is frozen. The next step is the complete ER diagram and exact database schema, followed by the full API specification — the point at which this architecture becomes actual code.